TALLER SEMANA 4

PRUEBAS PATRON ABSTRACT FACTORY

DOCENTE

ELIECER MONTERO OJEDA

INTEGRANTE

JUAN DAVID RENGIFO CHAVEZ

UNIDADES TECONOLOGICAS DE SANTANDER
PATRONES DE SOFTWARE
INGENIERIA EN SISTEMAS
2026

1. Introducción

El presente documento describe las pruebas funcionales realizadas sobre el módulo de gestión de ofertas de energía del sistema Enerlink, el cual implementa el patrón de diseño Abstract Factory para la creación de diferentes tipos de ventas de energía.

Dentro del sistema existen dos tipos de venta:

- Venta directa (DIRECT)
- Venta por subasta (AUCTION)

El patrón Abstract Factory permite que el sistema cree de manera consistente los objetos asociados a cada tipo de venta sin que la capa de aplicación dependa de clases concretas.

2. Objetivo de las pruebas

Validar el correcto funcionamiento del patrón Abstract Factory dentro del módulo de energía del sistema Enerlink, verificando que:

- El sistema crea correctamente ofertas de energía.
- El tipo de venta determina la fábrica concreta utilizada.
- Los endpoints REST funcionan correctamente.
- Las operaciones CRUD sobre las ofertas se ejecutan correctamente.

3. Entorno de pruebas

Las pruebas fueron realizadas en el siguiente entorno:

Componente	Descripción
Backend	Spring Boot
Lenguaje	Java
Arquitectura	Hexagonal
Herramienta de pruebas	Postman
URL base del servicio	http://localhost:8080
Endpoint principal	/api/offers

4. Endpoints del sistema

El controlador EnergyOfferController expone los siguientes endpoints para la gestión de ofertas de energía.

Método	Endpoint	Descripción
POST	/api/offers	Crear una nueva oferta
GET	/api/offers	Obtener todas las ofertas
GET	/api/offers/{id}	Obtener una oferta por ID
PUT	/api/offers/{id}	Actualizar una oferta
DELETE	/api/offers/{id}	Eliminar una oferta

5. Casos de prueba

5.1 Creación de oferta con venta directa

Esta prueba valida que el sistema utilice la fábrica **DirectSaleFactory** cuando el tipo de venta es **DIRECT**.

Request

- **Método**: POST
- **URL**: http://localhost:8080/api/offers

Body (JSON)

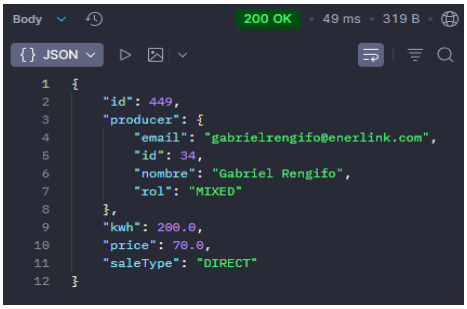
```
{
  "saleType": "DIRECT",
  "producerId": 34,
  "kwh": 200,
  "price": 70
}
```

Resultado esperado

El sistema debe:

1. Seleccionar la fábrica **DirectSaleFactory**.
2. Crear una instancia de **EnergyOffer** con tipo de venta DIRECT.
3. Ejecutar el proceso **DirectSaleProcess**.

Evidencia



```
Ejecutando venta directa...
```

Interpretación

Esta prueba demuestra que el patrón Abstract Factory permite crear una familia de objetos coherente para la venta directa sin que el servicio conozca la implementación concreta.

5.2 Creación de oferta mediante subasta

Esta prueba válida que el sistema utilice la fábrica **AuctionSaleFactory** cuando el tipo de venta es **AUCTION**.

Request

- **Método**: POST
- **URL**: http://localhost:8080/api/offers

Body

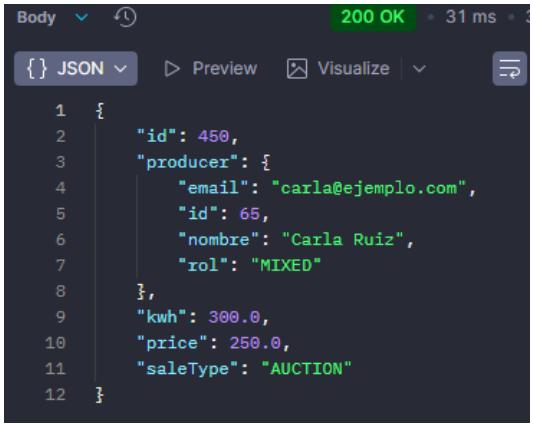
```
{
  "saleType": "AUCTION",
  "producerId": 65,
  "kwh": 300,
  "price": 250
}
```

Resultado esperado

El sistema debe:

1. Seleccionar la fábrica **AuctionSaleFactory**.
2. Crear una oferta configurada como subasta.
3. Ejecutar el proceso **AuctionSaleProcess**.

Evidencia



```
Ejecutando subasta...
```

Interpretación

La prueba evidencia que el sistema cambia automáticamente la familia de objetos creados según el tipo de venta, lo cual es el comportamiento esperado del patrón Abstract Factory.

5.3 Consulta de todas las ofertas

Esta prueba verifica que el sistema pueda recuperar todas las ofertas registradas.

Request

- **Método**: GET
- **URL**: http://localhost:8080/api/offers

Resultado esperado

El sistema devuelve una lista con todas las ofertas registradas.

Evidencia

Body 200 OK 52 ms 2.44 KB

{} JSON Preview Visualize


```
158    {
159        "id": 449,
160        "producer": {
161            "email": "gabrielrengifo@enerlink.com",
162            "id": 34,
163            "nombre": "Gabriel Rengifo",
164            "rol": "MIXED"
165        },
166        "kwh": 200.0,
167        "price": 70.0,
168        "saleType": "DIRECT"
169    },
170    {
171        "id": 450,
172        "producer": {
173            "email": "carla@ejemplo.com",
174            "id": 65,
175            "nombre": "Carla Ruiz",
176            "rol": "MIXED"
177        },
178        "kwh": 300.0,
179        "price": 250.0,
180        "saleType": "AUCTION"
181    }
```

5.4 Consulta de oferta por ID

Esta prueba permite verificar que el sistema pueda recuperar una oferta específica.

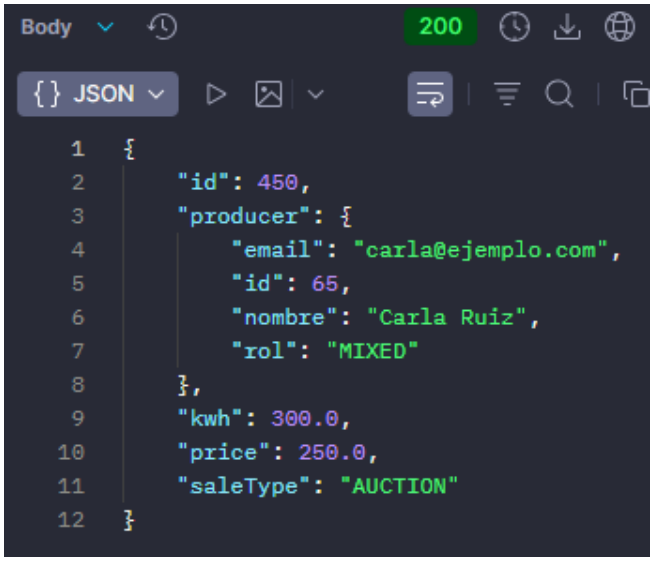
Request

- **Método**: GET
- **URL**: http://localhost:8080/api/offers/450

Resultado esperado

El sistema retorna la información de la oferta solicitada.

Evidencia



5.5 Actualización de oferta

Esta prueba válida la actualización de los datos de una oferta existente.

Request

- **Método**: PUT
- **URL**: http://localhost:8080/api/offers/450

Body

```
{
  "saleType": "AUCTION",
  "producerId": 65,
  "kwh": 500,
  "price": 175
}
```

Resultado esperado

La oferta debe actualizar sus valores de energía y precio.

Evidencia


```
Body  200
{} JSON
1  {
2      "id": 450,
3      "producer": {
4          "email": "carla@ejemplo.com",
5          "id": 65,
6          "nombre": "Carla Ruiz",
7          "rol": "MIXED"
8      },
9      "kwh": 500.0,
10     "price": 175.0,
11     "saleType": "AUCTION"
12 }
```


5.6 Eliminación de oferta

Esta prueba verifica que el sistema pueda eliminar una oferta existente.

Request

Método: DELETE

URL: http://localhost:8080/api/offers/450

Resultado esperado

El sistema elimina la oferta y retorna un código HTTP **204 No Content**.

Evidencia


```
Body  204 No Content • 66 ms • 112 B
```

6. Resultados de las pruebas

Las pruebas realizadas permitieron verificar que el sistema Enerlink implementa correctamente el patrón Abstract Factory, ya que la creación de ofertas y procesos de venta depende exclusivamente del tipo de venta especificado.

El servicio de aplicación no depende de clases concretas como DirectSaleProcess o AuctionSaleProcess, sino que delega la creación de estos objetos a las fábricas correspondientes.

Esto confirma que el sistema cumple con los principios de diseño asociados al patrón, especialmente:

- Open/Closed Principle
- Dependency Inversion Principle
- Single Responsibility Principle

7. Conclusiones

La implementación del patrón Abstract Factory dentro del módulo de energía del sistema Enerlink permite desacoplar la lógica de creación de objetos del servicio de aplicación.

Gracias a esta arquitectura, el sistema puede soportar múltiples tipos de venta de energía manteniendo consistencia en la creación de sus objetos asociados.

Además, la solución es fácilmente extensible, ya que nuevos tipos de venta podrían agregarse mediante nuevas fábricas concretas sin modificar el código existente del servicio.

Las pruebas realizadas mediante Postman evidencian que el sistema responde correctamente a las diferentes solicitudes y selecciona dinámicamente la fábrica adecuada según el tipo de venta.